Finding errors

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap(int *a, int *b);
5  void printArray(int *arr, int size);
6
7  int main()
8  {
9      int x = 10;
10     int y = 20;
11
12     int *ptr1;
13
14     printf("%d\n", *ptr1);
15     // Error: Dereferencing an uninitialized pointer causes undefined behavior.
16     // Solution: Initialize ptr1 before dereferencing, e.g., ptr1 = &x;
17
18     int *ptr2 = NULL;
19
20     *ptr2 = 100;
21     // Error: Dereferencing a NULL pointer causes a runtime crash.
22     // Solution: Allocate memory or assign a valid address before dereferencing.
23
24     int *ptr3;
25
26     ptr3 = &x;
```

```
27
28      printf("%d\n", *ptr3);
29
30      ptr3++;
31      // Warning: Pointer increment moves ptr3 beyond variable x, causing invalid
access.
32      // Solution: Do not perform pointer arithmetic on a pointer to a single variable.
33
34      printf("%d\n", *ptr3);
35      // Error: Dereferencing an invalid pointer after incrementing past x.
36      // Solution: Access only valid memory locations.
37
38      int arr[5] = {10,20,30,40,50};
39
40      int *ptr4 = arr;
41
42      printf("%d\n", *(ptr4 + 10));
43      // Error: Accessing memory beyond array bounds.
44      // Solution: Use indices within 0 to 4.
45
46      int *ptr5;
47
48      ptr5 = malloc(sizeof(int));
49
50      *ptr5 = 500;
51
52      printf("%d\n", *ptr5);
53
```

```
54    free(ptr5);
55
56    printf("%d\n", *ptr5);
57    // Error: Dereferencing a pointer after free() (dangling pointer).
58    // Solution: Do not access memory after free(); set ptr5 = NULL.
59
60    int *ptr6;
61
62    ptr6 = malloc(sizeof(int));
63
64    *ptr6 = 1000;
65
66    ptr6 = NULL;
67    // Warning: Memory leak. Allocated memory is lost without free().
68    // Solution: free(ptr6) before assigning NULL.
69
70    int *ptr7;
71
72    ptr7 = malloc(5);
73    // Warning: Allocates only 5 bytes, which may be insufficient for integer array
usage.
74    // Solution: ptr7 = malloc(5 * sizeof(int));
75
76    for(int i=0; i<10; i++)
77    {
78        ptr7[i] = i;
79        // Error: Writing beyond allocated memory.
80        // Solution: Allocate enough memory and keep index within bounds.
```

```
81  }
82
83  free(ptr7);
84
85  int *ptr8;
86
87  ptr8 = malloc(3 * sizeof(int));
88
89  ptr8[0] = 10;
90  ptr8[1] = 20;
91  ptr8[2] = 30;
92  ptr8[3] = 40;
93  // Error: Array index 3 is out of bounds for allocation of 3 integers.
94  // Solution: Allocate 4 integers or remove this assignment.
95
96  free(ptr8);
97
98  int *ptr9;
99
100  ptr9 = malloc(sizeof(int));
101
102  *ptr9 = 50;
103
104  free(ptr9);
105
106  free(ptr9);
107  // Error: Double free causes undefined behavior.
108  // Solution: Set ptr9 = NULL after first free and avoid freeing twice.
```

```
109
110   int *ptr10;
111
112   ptr10 = malloc(sizeof(int));
113
114   printf("%d\n", *ptr10);
115   // Warning: Reading uninitialized allocated memory.
116   // Solution: Initialize *ptr10 before use.
117
118   int *numbers;
119
120   numbers = malloc(5 * sizeof(int));
121
122   for(int i=0; i<=5; i++)
123   // Error: Loop writes to numbers[5], which is out of bounds.
124   // Solution: for(int i=0; i<5; i++)
125   {
126      numbers[i] = i * 10;
127   }
128
129   printArray(numbers,5);
130
131   int a = 100;
132   int b = 200;
133
134   swap(&a,&b);
135
136   printf("%d %d\n", a, b);
```

```
137  // Warning: swap() implementation does not actually swap caller values.
138  // Solution: Correct swap() using value assignment through pointers.
139
140  int **pptr;
141
142  int value = 500;
143
144  pptr = &value;
145  // Error: Incompatible pointer types. pptr expects int ** but &value is int *.
146  // Solution: Use int *ptr = &value; pptr = &ptr;
147
148  printf("%d\n", **pptr);
149  // Error: Invalid double dereference due to incorrect pointer assignment.
150  // Solution: Assign pptr to a valid int ** object.
151
152  char *name;
153
154  name = malloc(5);
155
156  strcpy(name,"Programming");
157  // Error: Buffer overflow. Destination buffer is too small.
158  // Solution: Allocate enough memory, e.g., malloc(strlen("Programming")+1).
159
160  printf("%s\n", name);
161
162  int *ptr11;
163
164  ptr11 = malloc(sizeof(int));
```

```
165
166   *ptr11 = 123;
167
168   ptr11++;
169   // Warning: Pointer no longer points to allocated object.
170   // Solution: Do not increment pointer beyond allocated memory.
171
172   printf("%d\n", *ptr11);
173   // Error: Dereferencing invalid pointer after increment.
174   // Solution: Access only valid allocated memory.
175
176   int *ptr12 = NULL;
177
178   if(*ptr12 > 0)
179   // Error: Dereferencing NULL pointer.
180   // Solution: Check ptr12 != NULL before dereferencing.
181   {
182      printf("Positive\n");
183   }
184
185   int *ptr13;
186
187   ptr13 = malloc(100 * sizeof(int));
188
189   for(int i=0;i<100;i++)
190   {
191      ptr13[i] = i;
192   }
```

```
193
194     ptr13[100] = 500;
195     // Error: Array index 100 is out of bounds.
196     // Solution: Use indices 0 to 99 only.
197
198     int *ptr14;
199
200     ptr14 = malloc(sizeof(int));
201
202     free(ptr14);
203
204     *ptr14 = 999;
205     // Error: Writing to freed memory (use-after-free).
206     // Solution: Do not access ptr14 after free(); set ptr14 = NULL.
207
208     int matrix[3][3];
209
210     int *p;
211
212     p = matrix;
213     // Warning: Incompatible pointer assignment from 2D array to int *.
214     // Solution: Use int (*p)[3] = matrix;
215
216     printf("%d\n", *(p + 20));
217     // Error: Accessing memory far beyond matrix bounds.
218     // Solution: Access only valid matrix elements.
219
220     int *ptr15;
```

```
221
222     ptr15 = malloc(sizeof(int));
223
224     ptr15 = malloc(sizeof(int));
225     // Warning: Memory leak. Previous allocated memory is lost.
226     // Solution: free(ptr15) before reassigning or reuse existing allocation.
227
228     printf("End of Program\n");
229
230     return 0;
231 }
232
233 void swap(int *a, int *b)
234 {
235     int *temp;
236
237     temp = a;
238     a = b;
239     b = temp;
240     // Error: Swaps local pointer copies, not the values pointed to.
241     // Solution:
242     // int temp = *a;
243     // *a = *b;
244     // *b = temp;
245 }
246
247 void printArray(int *arr, int size)
248 {
```

```
249    for(int i=0; i<=size; i++)
250    // Error: Loop accesses arr[size], which is out of bounds.
251    // Solution: for(int i=0; i<size; i++)
252    {
253        printf("%d ", arr[i]);
254    }
255
256    printf("\n");
257 }
```